

Arquitetura do Sync

O Sync é a camada de integração que faz o contexto do atendimento viajar junto com o cliente. A conversa começa no site, continua no WhatsApp ou com um atendente, e ninguém repete nada. Este documento descreve o que foi construído, como as peças se ligam e o que está simulado.

PROCESSOS	MÓDULOS	TABELAS	TESTES
1 API + 3 fronts	9	11	321

Panorama

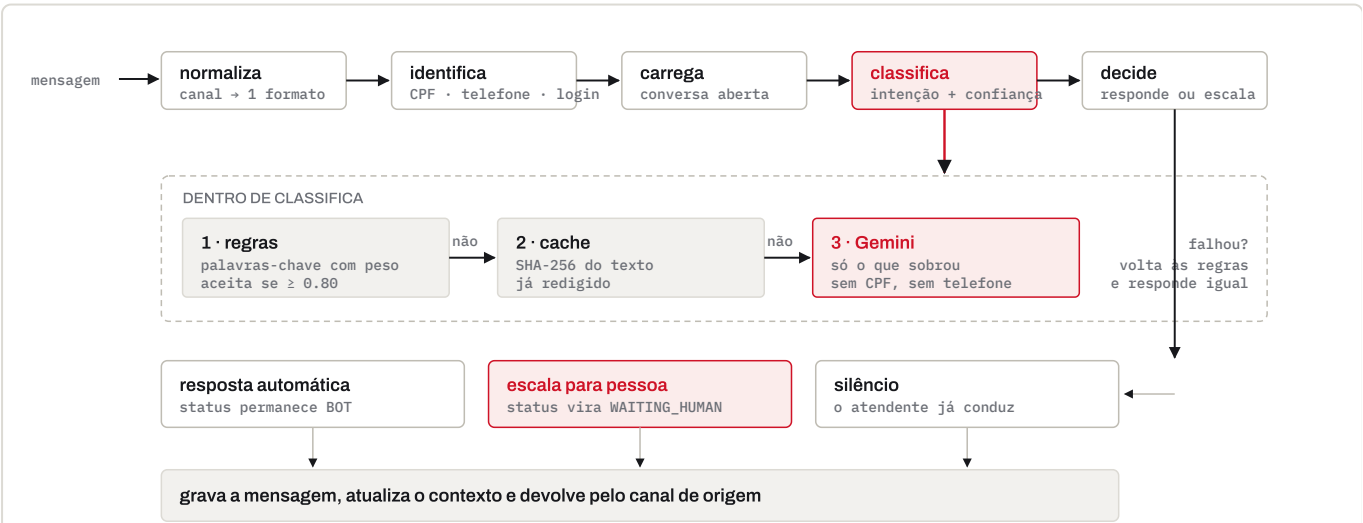
Um **monolito modular**: um único processo Node, com cada caixa do diagrama isolada atrás da própria interface TypeScript. A escolha é deliberada. Microsserviços dariam ao projeto a complexidade de operação de um sistema grande sem nenhum dos problemas que a justificam, e separar um módulo em serviço depois é mecânico, porque a fronteira já existe no tipo.



channels Camada Sync. Traduz site, app e WhatsApp para um único formato de mensagem.	identity Resolve o cliente por login, CPF, telefone ou protocolo.
nlp Regras, cache e Gemini. Remove dados pessoais antes de qualquer chamada externa.	conversation Orquestrador, política de escalonamento, respostas automáticas e handoff.
auth Login de cliente e de atendente, Argon2id, JWT, rotação de refresh, papéis.	admin Fila por intenção, atendimento, métricas e desempenho por pessoa.
insights Sugestão de oferta e título do cartão, ambos escritos pela IA.	context Repositórios Prisma. É o único lugar que conhece o banco.
gateway Rotas Express, validação Zod e a raiz de composição do sistema.	

O caminho de uma mensagem

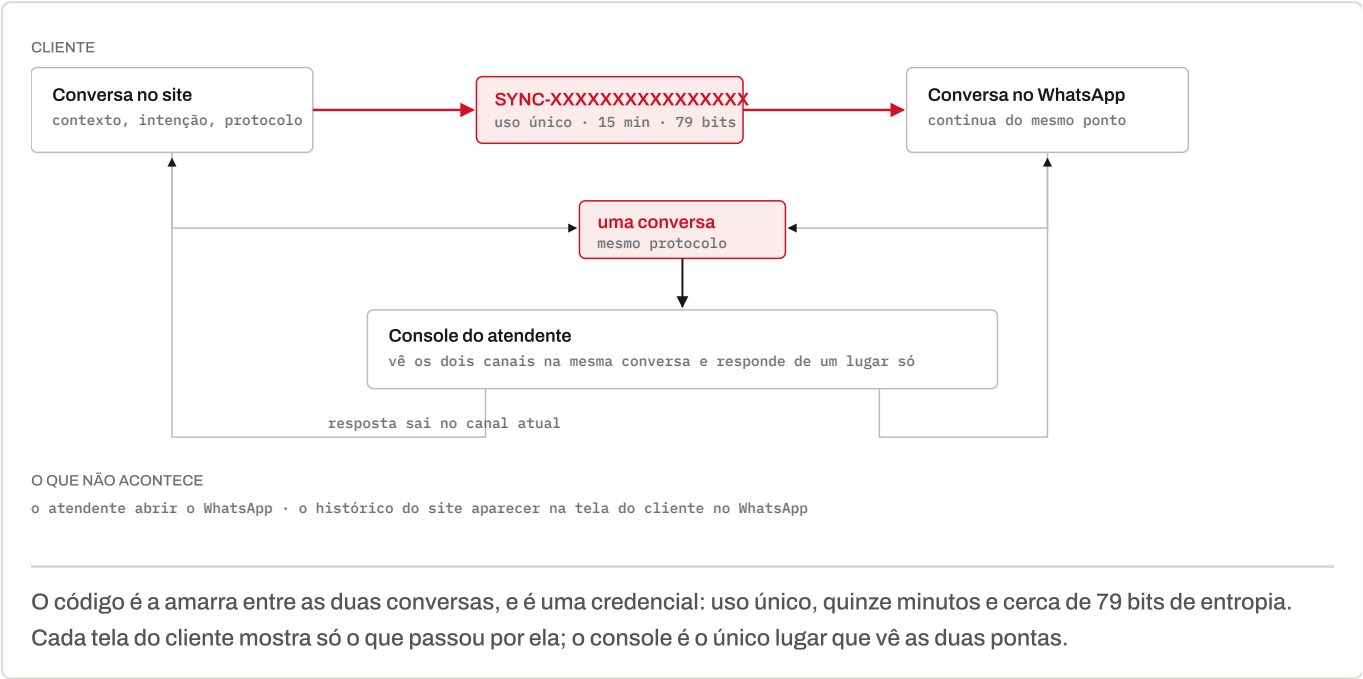
Toda mensagem entra pelo mesmo pipeline, venha do site ou do WhatsApp. A ordem importa: a identificação vem antes da classificação porque o contexto muda a resposta, e a persistência da mensagem vem depois da classificação para a linha nascer já com intenção e confiança.



A cascata de classificação é o que mantém a demonstração de pé: as regras resolvem a maioria sozinhas, o cache evita pagar duas vezes pela mesma frase, e o modelo entra apenas no que sobrou. Se ele falhar ou a cota acabar, o sistema continua respondendo pelas regras.

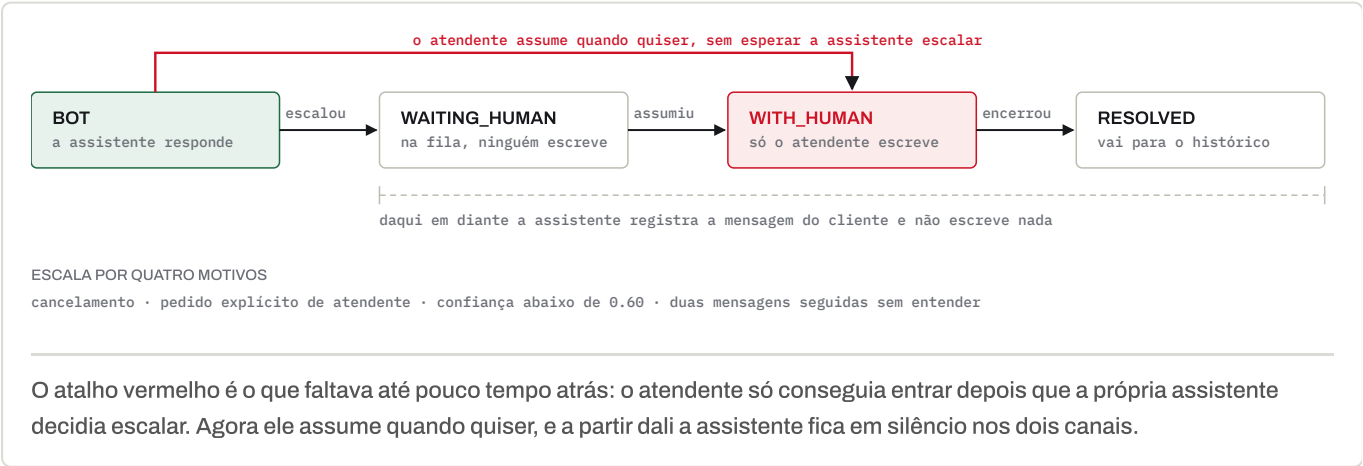
Continuidade entre canais

Este é o ponto do produto, e é onde a arquitetura foi mais deliberada. O cliente muda de canal; **o atendente não muda de ferramenta**. Ele continua dentro do Sync, que mostra o site e o WhatsApp na mesma conversa. É o modelo de caixa de entrada unificada, o mesmo que ferramentas como o ManyChat usam sobre a API oficial da Meta.



Quem fala com o cliente

A conversa tem quatro estados, e eles decidem quem responde. A regra que governa tudo é simples: **assumido pelo atendente, a assistente para de responder**, nos dois canais. Duas vezes na mesma conversa se contradizem.



Funcionalidades

Para o cliente

O acesso tem três caminhos: entrar, primeiro acesso e recuperação de senha. Não existe cadastro aberto, porque uma conta criada do zero não teria plano nem fatura sobre o que conversar. Quem já é cliente prova quem é confirmando CPF e e-mail, e define a senha.

No chat, uma faixa no topo mostra **o que o Sync já sabe**: quem é a pessoa, o canal, o assunto, o serviço e o protocolo, marcando com um visto conforme identifica. Isso é também transparência sob a LGPD, porque a pessoa vê exatamente qual dado dela a conversa guarda. O telefone é pedido depois da primeira troca, e não na entrada, para o atendimento não virar cadastro. É esse número que liga o site ao WhatsApp quando não há login.

Para o atendente

A fila não é uma tabela com filtro: cada coluna é a intenção pela qual a pessoa procurou a Claro, em ordem de custo de ignorar. O título de cada cartão é escrito pela IA a partir do que o cliente disse; sem IA disponível, o cartão mostra a mensagem crua. Cada atendimento tem endereço próprio e abre em outra aba, com a fila continuando atualizada atrás.

Ao lado da conversa, o Sync sugere **o que faria sentido oferecer àquele cliente**, com a justificativa e o grau de confiança. A sugestão nasce de regras sobre o perfil, e o modelo escreve o texto quando está disponível. Quando há problema técnico aberto, ela diz para resolver a falha antes de oferecer qualquer coisa.

Para a gestão

O menu muda com o papel. Em vez de "meu painel", a gestão vê a equipe inteira e entra no painel de cada pessoa. As permissões são uma **tabela de dados no servidor**, não uma condição espalhada pela interface, e o papel é lido do banco a cada requisição, nunca do token, para que promover ou rebaixar alguém valha na hora.

Segurança e dados pessoais

DECISÃO	COMO	POR QUÊ
Senhas	Argon2id, salt por senha embutido no hash	Recomendação do OWASP. Vazar o banco não entrega senhas.
Sessão	JWT de 15 minutos e refresh opaco que gira a cada uso	O refresh precisa ser revogável, e um JWT autocontido não seria.
Roubo de sessão	Detecção de reuso: usar um refresh já gasto revoga a família inteira	É o sinal de que o token foi copiado.
Dados pessoais na IA	CPF, telefone e e-mail viram marcadores antes de qualquer chamada	As entidades reais são extraídas aqui por regex. Nada disso chega ao Google.
CPF na tela	Mascarado para o atendente: ***.456.789-**	Atender não exige ver o documento inteiro.
Código de continuidade	16 caracteres, uso único, 15 minutos, 30 tentativas por minuto na porta	É uma credencial: quem acertar um entra na conversa de outra pessoa.
Conversa alheia	Cliente autenticado só alcança a própria conversa	Antes, um login qualquer mais o id abria a conversa de outro cliente.
Telefone digitado	Não concede identidade nenhuma, só reencontra a conversa	Telefone não é segredo. Digitar o número de outro não pode revelar a fatura dele.

Dados

Onze tabelas em MySQL 8, acessadas só pelos repositórios do módulo `context`. O atendimento é a `Conversation`: ela carrega o protocolo, o

canal de origem, o canal atual, a intenção, o estado e o telefone de contato, que é a chave que atravessa os canais.

TABELA	GUARDA
Conversation	O atendimento: protocolo, canais, intenção, estado, responsável, telefone de contato e o título escrito pela IA.
Message	Cada fala, com o canal em que aconteceu. É esse campo que permite a cada tela do cliente mostrar só o que passou por ela.
Customer	Cliente da Claro: CPF, nome, e-mail, telefone e o hash da senha.
Service e Invoice	Plano e fatura. Alimentam a resposta automática e a sugestão de oferta.
Agent	A equipe, com o papel de atendimento ou gestão.
HandoffToken	O código de continuidade, com validade e marca de uso.
RefreshToken	Só o hash, com família e marca de uso, para detectar reuso.
PasswordResetToken	Recuperação de senha. Também só o hash.
IntentCache	Classificação já feita, indexada pelo hash do texto redigido.
OfferInsight	A sugestão de oferta, com origem e confiança.

O que é real e o que entraria no lugar

Back-end e front-end rodam de verdade contra um MySQL real. Três coisas estão simuladas, e cada uma tem um ponto de troca explícito no código, não uma reescrita.

PEÇA	HOJE	COMO ESTÁ	PROD	EM PRODUÇÃO
WhatsApp	Driver <code>mock</code> : uma tela React que fala com a mesma API, informando canal e telefone. A porta dela existe só neste modo.		Driver <code>meta</code> : Cloud API, número dedicado no Meta Business Suite e webhook assinado, respeitando a janela de 24 horas e os templates. O back-end não sabe a diferença.	
E-mail e SMS	Não há remetente. O código de recuperação aparece na tela, marcado como demonstração, e vai para o log do servidor.		O mesmo código sai por e-mail ou SMS. A tela é idêntica; o campo já está lá esperando o que a pessoa recebeu.	
Base de clientes	Semeada no MySQL, com dois clientes e quatro pessoas na equipe.		Cadastro real da Claro. O primeiro acesso continua igual.	
Modelo de linguagem	Gemini na camada gratuita, chamado só quando as regras não bastam.		Cota paga e modelo dedicado. Se ele cair, as regras seguem respondendo sozinhas.	
Telefone informado	Não verificado. Serve só para reencontrar a conversa.		Confirmação por SMS antes de gravar o número.	

Como rodar

Um comando sobe o banco no Docker, semeia a base e liga os quatro processos: `npm run start:all` . A API responde em `:3333` , o site em `:5173` , o console em `:5174` e o WhatsApp simulado em `:5175` . A suíte tem **321 testes**, e roda contra o banco de verdade em vez de mocks de repositório.